

AuChimiste Tutorial

Walter Dal'Maz Silva

2026-07-27

Let's start by a global import; everything that is intended to be accessible to the end user is found here:

```
using Pkg
open("pkg_init.log", "w") do log
    Pkg.activate(joinpath(@__DIR__, "julia-shared")); io=log
end

push!(LOAD_PATH, joinpath(@__DIR__, "scripts"))
import AuChimiste as AC
```

Elements database

A built-in elements database is provided by `ChemicalElements`, which is the base building block of `AuChimiste`. It is an extremely simple module and below we go through the whole of its exposed functionalities in just a few lines of code

i Database extent

In general you only need to worry about using `ChemicalElements` directly if your calculations require isotopes to be added. The default table of elements provides access only to stable elements.

You can get a list of available atomic symbols with `list_elements`. Suppose you want to check if deuterium *D* is present in the list, you can use its symbol for inspection:

```
false
```

Since it is not present but your calculations require this isotope, you feed the database with `add_element`; you also decide to add tritium. In fact `add_element` will not fail if the element exists, but issue a warning. You can try adding an existing element to see what happens:

```
└ Warning: The provided element H is already present in the elements dictionary.  
If you are trying to create an isotope, please chose a different name. Also  
notice that default stable elements cannot be modified.
```

```
L                                     @                                     AuChimiste  
F:\kompanion\repos\wallytutor.github.io\shorts\scripts\AuChimiste.jl:273
```

If you wish to get back to the standard data you can do so with `reset_elements_table`. Notice below that deuterium mass is no longer available.

```
false
```

Element data retrieval

It is possible to retrieve the `atomic_mass`. Other data retrieval functions include `atomic_number` and `element_name`. All of these work with both string and symbols.

```
AC.atomic_mass(:C), AC.atomic_number(:C), AC.element_name(:C)
```

```
(12.011, 6, "carbon")
```

Getting the whole `element` data can be achieved at once as follows:

```
AC.element(:Cl)
```

```
Cl (17, chlorine) 35.45 kg/kmol
```

In this case is also possible to query the data through the atomic number:

```
AC.element(26)
```

```
Fe (26, iron) 55.845 kg/kmol
```

On the other hand atomic masses of unstable elements are not accessible:

```
try AC.atomic_mass(:Po) catch e; @error(e) end
```

```
└ Error: NoIsotopeProvidedError: Accessing the atomic mass of unstable element  
Po is not supported. Please consider creating a named isotope of this element
```

```
with `add_isotope`.
↳ @ Main In[37]:1
```

To have an unstable element listed, you need to `add_isotope` before. For instance, let's add Po-187 to the database.

```
AC.add_isotope(:Po, 187.003030)
```

```
Po187 (84, polonium-187) 187.00303 kg/kmol
```

Creating components

Component creation is a trivial task with `AuChimiste`. All you need to do is call `component` with one of the available composition specification methods and a list of keyword arguments representing the element amounts to use. For instance, to create aluminum oxide from its stoichiometry one does:

```
A = AC.component(:stoichiometry; Al=2, O=3)
```

```
AuChimiste.ChemicalComponent([:Al, :O], [2.0, 3.0], [0.4, 0.6],
[0.5292569257852893, 0.47074307421471057], 101.9600768, 0)
```

The above is a syntactic sugar to providing a `stoichiometry` as argument. Please notice that this form requires the overall charge of the component to be specified.

```
A = AC.component(AC.stoichiometry(Al=2, O=3), 0)
```

```
AuChimiste.ChemicalComponent([:Al, :O], [2.0, 3.0], [0.4, 0.6],
[0.5292569257852893, 0.47074307421471057], 101.9600768, 0)
```

The other composition specification methods are `mass_proportions` and `mole_proportions`. Let's see their use in a more elaborate example with naphthalene $C_{10}H_8$:

```
naphtalene = AC.component(:stoichiometry; C=10, H=8)
```

```
AuChimiste.ChemicalComponent([:C, :H], [10.0, 8.0], [0.5555555555555556,
0.4444444444444444], [0.937085524365316, 0.06291447563468411],
128.17399999999998, 0)
```

So far nothing new. We can use `mass_fractions_map` and `mole_fractions_map` to retrieve the named-tuples of compositions in the units provided by these functions:

```
Y = AC.mass_fractions_map(naphtalene)
X = AC.mole_fractions_map(naphtalene)
Y, X
```

```
((C = 0.937085524365316, H = 0.06291447563468411), (C = 0.5555555555555556, H
= 0.4444444444444444))
```

Using the mass fractions `Y` one can create the equivalent compound from this sort of input. Notice here that a scale is provided to enforce the stoichiometric coefficient of carbon in the species (there is no way to infer it simply from elemental mass fractions).

```
m_y = AC.component(:mass_proportions; Y..., scale=:C=>10)
```

```
AuChimiste.ChemicalComponent([:C, :H], [10.0, 8.0], [0.5555555555555556,
0.4444444444444444], [0.937085524365316, 0.06291447563468411],
128.17399999999998, 0)
```

The same can be done using mole fractions, as follows:

```
m_x = AC.component(:mole_proportions; X..., scale=:C=>10)
```

```
AuChimiste.ChemicalComponent([:C, :H], [10.0, 8.0], [0.5555555555555556,
0.4444444444444444], [0.937085524365316, 0.06291447563468411],
128.17399999999998, 0)
```

If scaling is not provided, the default behavior is enforced, applying unit content to the first element provided in the composition tuple. Notice that these constructors do not sort arguments. This behavior is intended so that compounds can be easily understood by the user in the case a standard formula format exists (as for the oxides above).

```
AC.component(:mole_proportions; X...)
```

```
AuChimiste.ChemicalComponent([:C, :H], [0.5555555555555556,
0.4444444444444444], [0.5555555555555556, 0.4444444444444444],
[0.937085524365316, 0.06291447563468411], 7.120777777777777, 0)
```

Finally, by *proportions* instead of *fractions* in the name of the composition specification methods we mean that internal normalization is performed. That might be useful, for instance, for reverse-engineering a compound formula from an analysis report.

Often in the field of combustion of heavy-fuel oils one is confronted with empirical fuel compositions given in mass percentages. Assume the following composition; playing with the scaling factor an engineer could infer a candidate composition and identify the family of the fuel [1].

```
AC.component(:mass_proportions; C = 93.2, H = 6.3, O = 0.3, scale=:C=>10)
```

```
AuChimiste.ChemicalComponent([:C, :H, :O], [10.0, 8.054586909871242,
0.024165271059054916], [0.5531355206333394, 0.445527812387811,
0.001336666978849611], [0.9338677354709419, 0.06312625250501001,
0.003006012024048096], 128.61564377682402, 0)
```

Combining components

Some algebraic manipulation is also possible with `ChemicalComponent` instances. Let's see a practical case from cement industry, where compositions are often expressed with a jargon that makes use of multiple of component oxides to represent complex phases such as $C_{12}A_7$.

Below we create a component `C` for calcium oxide (notice that `C` here was chosen as per industry jargon, it has nothing to do with carbon) and create the multiples of the base oxides (using an extension of `Base.*`) before combining them through addition (again, by extending `Base.:+` operator).

```
A = AC.component(:stoichiometry; Al=2, O=3)
C = AC.component(:stoichiometry; Ca=1, O=1)
```

```
C12A7 = 12C + 7A
```

```
AuChimiste.ChemicalComponent([:Al, :Ca, :O], [14.0, 12.0,
33.0], [0.23728813559322035, 0.2033898305084746, 0.559322033898305],
[0.2724141100024047, 0.3468343810969771, 0.38075150890061815],
1386.6445376000001, 0)
```

⚠ Meaning of operations

All operations performed over `ChemicalComponent` instances are defined on stoichiometric coefficients, *i.e.* the scaling provided by multiplication acts directly on those coefficients, while combining will add coefficients for corresponding elements.

Subtraction operation (`Base.:-`) is also possible, but there are many conditions under which it could fail and it was chosen by design that a negative composition should return the mass imbalance instead, *i.e.* what lacks in `A` to be subtracted `C` and still return a valid component.

```
A = AC.component(:stoichiometry; Al=2, O=3)
C = AC.component(:stoichiometry; Ca=1, O=1)
A - C
```

```
└ Warning: Component subtraction is fragile and must be used with care. The
component operation you tried to perform produced negative coefficients. That
is because the right component have more of one/some elements than the left
component. Instead of retuning the meaningless composition I am providing you
with the mass imbalance.
```

```
L                                     @                               AuChimiste
F:\kompanion\repos\wallytutor.github.io\shorts\scripts\AuChimiste.jl:861
```

```
AuChimiste.ChemicalComponent([:Ca], [1.0], [1.0], [1.0], 40.078, 0)
```

On the other hand, if the left component has *enough* of what is being subtracted by the right component, then the actual resulting component is returned:

```
C12A7 - C
```

```
AuChimiste.ChemicalComponent([:Al,      :Ca,      :O],      [14.0,      11.0,
32.0],      [0.24561403508771928,      0.19298245614035087,      0.5614035087719298],
[0.2838950499884795,      0.3313307949742964,      0.3847741550372241],
1330.5675376000002, 0)
```

Charged components

For the sake of generality, charged components are also supported. This is required so that ions can be considered in reactions. For instance, to create a sodium chloride formula one can proceed as follows:

```
Na+ = AC.component(:stoichiometry; charge=+1, Na=1)
Cl- = AC.component(:stoichiometry; charge=-1, Cl=1)

NaCl = Na+ + Cl-
```

```
AuChimiste.ChemicalComponent([:Cl, :Na], [1.0, 1.0], [0.5, 0.5],
[0.6066074598985822, 0.3933925401014177], 58.43976928000001, 0)
```

The user must be aware of a few caveats; first, molecular mass is not corrected for the missing/extra electrons. Electron and mass balance are handled separately, as this makes more sense from a physical standpoint. Scaling of a component also scales its charge. This is required for proper balance of chemical equations.

```
(3Na+).charge
```

```
3
```

So the formation of calcium hydroxide can be stated as:

```
Ca+2 = AC.component(:stoichiometry; charge=+2, Ca=1)
OH- = AC.component(:stoichiometry; charge=-1, O=1, H=1)

Ca+2 + 2OH-
```

```
AuChimiste.ChemicalComponent([:Ca, :H, :O], [1.0, 2.0, 2.0], [0.2, 0.4, 0.4],
[0.5409220968525618, 0.027209415321492198, 0.4318684878259461], 74.092, 0)
```

Quantities of matter

An arbitrary amount of matter can be constructed with `quantity`. The resulting `ComponentQuantity` object supports both scaling (multiplication) and additive (summation) operations. A trivial example would be:

```
3AC.quantity(A, 1.0)
```

```
AuChimiste.ComponentQuantity(3.0, AuChimiste.ChemicalComponent([:Al, :O], [2.0,
3.0], [0.4, 0.6], [0.5292569257852893, 0.47074307421471057], 101.9600768, 0))
```

⚠ Meaning of operations

On the other hand, operations performed on `ComponentQuantity` entities are scaled by the elemental mass fractions. That is mostly intuitive in the context of application of this structure.

Well, there is nothing special there, the mass was scaled by *three* with no composition change. The next example is maybe more instructive: we mix one mole of `A` with one mole of `C` by providing their molar masses as the mass of each component. This is interesting because one can quickly verify the correctness of the results.

```
ma = AC.quantity(A, 0.001A.molar_mass)
mc = AC.quantity(C, 0.001C.molar_mass)
ma + mc
```

```
AuChimiste.ComponentQuantity(0.1580370768,
AuChimiste.ChemicalComponent([:Al, :Ca, :O], [0.05396307679999999,
0.0269815384, 0.10792615359999998], [0.2857142857142857, 0.14285714285714288,
0.5714285714285714], [0.34145833302327955, 0.2535987175384151,
0.4049429494383055], 4.264083456302949, 0))
```

Because in many situations one may be interested in mixing quantities directly, a wrapper is provided for eliminating the need of and explicit creation of a component.

```
ma = AC.quantity(:stoichiometry, 1.0; Al=2, O=3)
mc = AC.quantity(:stoichiometry, 1.0; Ca=1, O=1)
ma + mc
```

```
AuChimiste.ComponentQuantity(2.0, AuChimiste.ChemicalComponent([:Al, :Ca, :O],
[0.5292569257852893, 0.48115160226117654, 1.2750369909391104],
[0.2315771351312748, 0.21052858108590652, 0.5578942837828187],
[0.2646284628926447, 0.3573479323073631, 0.37802360479999225],
53.963076799999996, 0))
```

 Charge operations

Operations on electrical charges of quantities of matter are currently not supported. This functionality was initially conceived for balancing macroscopic amounts of matter and may in the future be extended for ionic substances.

Bibliography

- [1] C. J. Lawn, *Principles of combustion engineering for boilers*. in Combustion treatise. Academic Press, 1987.